NAME:ADITYA BABAR
COURSE:DATA ANALYTICS

HR & Payroll Management System — SQL-Only Project

1. Project Overview

This project simulates a real-world Human Resource & Payroll System for a company.

Students will design the database, insert data, run analytical queries, create views, and solve HR-related case studies using SQL only.

Concepts Covered:

- DDL (CREATE, ALTER, DROP)
- DML (INSERT, UPDATE, DELETE)
- Constraints
- Joins (INNER, LEFT, RIGHT, FULL)
- Group By & Having
- Aggregate Functions - > Adv Part
- Subqueries & Nested Queries
- Store procedures
- Views

2. ER Diagram

```
+-------------------+
|    DEPARTMENTS    |
|-------------------|
| PK dept_id        |
| dept_name         |
+-------------------+
          |
          | 1
          |
          |---------------------------+
                                      |
                                      | M
                              +-------------------+
                              |     EMPLOYEES     |
                              |-------------------|
                              | PK emp_id         |
                              | emp_name          |
                              | dept_id (FK)      |
                              +-------------------+
                                      |
```

```
                                      |
           +-------------------------+-------------------------+
           |                         |                         |
           | 1                       | 1                       | 1
           |                         |                         |
           | M                       | M                       | M
+-------------------+      +-------------------+      +-------------------+
|     SALARIES      |      |    ATTENDANCE     |      | EMPLOYEE_PROJECTS |
|-------------------|      |-------------------|      |-------------------|
| PK salary_id      |      | PK attendance_id  |      | PK (emp_id,       |
| emp_id (FK)       |      | emp_id (FK)       |      |     project_id)   |
| salary_amount     |      | date              |      | emp_id (FK)       |
+-------------------+      +-------------------+      | project_id (FK)   |
                                                      +-------------------+
                                                               |
                                                               | M
                                                               |
                                                               | 1
                                                      +-------------------+
                                                      |      PROJECTS     |
                                                      |-------------------|
                                                      | PK project_id     |
                                                      | project_name      |
                                                      +-------------------+
```

A. Basic Queries

[1]List all employees

Code:

SELECT * FROM employees;

Screenshot:

Result Grid | Filter Rows: | Edit: | Expor

emp_id	emp_name	email	phone	hire_date	d
101	Amit Sharma	amit@company.com	9876543210	2022-01-10	2
102	Priya Singh	priya@company.com	9922334455	2021-03-15	1
103	Rahul Verma	rahul@company.com	8811223344	2020-07-22	2
104	Neha Joshi	neha@company.com	9988776655	2023-02-10	4
105	Karan Patel	karan@company.com	9090909090	2019-11-01	3

[2]Employees hired after 2021

Code:

select * from employees

where hire_date>2021;

Screenshot:

Result Grid | Filter Rows: | Edit: | Expor | Result Grid

emp_id	emp_name	email	phone	hire_date	d
101	Amit Sharma	amit@company.com	9876543210	2022-01-10	2
102	Priya Singh	priya@company.com	9922334455	2021-03-15	1
103	Rahul Verma	rahul@company.com	8811223344	2020-07-22	2
104	Neha Joshi	neha@company.com	9988776655	2023-02-10	4
105	Karan Patel	karan@company.com	9090909090	2019-11-01	3

[3]HR department employees

Code:

```
select e.* from employees e
join departments d
on e.dept_id=d.dept_id
where d.dept_name="HR";
```

Screenshot:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	email	phone	hire_date	dept_id	job_role
102	Priya Singh	priya@company.com	9922334455	2021-03-15	1	HR

B. Filtering & Sorting

[1]. Employees with salary > 50,000

Code:

```
select e.emp_name,s.basic_salary from employees e
join salaries s
on e.emp_id=s.emp_id
where s.basic_salary >5000;
```

Screenshot:

Result Grid | Filter Rows:

emp_name	basic_salary
Amit Sharma	60000.00
Priya Singh	45000.00
Rahul Verma	80000.00
Neha Joshi	40000.00
Karan Patel	55000.00

[2]. Order employees by hire_date

Code:

```
select hire_date from employees

order by hire_date asc;
```

Screenshot:

hire_date
2019-11-01
2020-07-22
2021-03-15
2021-04-19
2022-01-10

#—-0r—---

```
#In descending order

select hire_date from employees

order by hire_date desc;
```

Screenshot:

Result Grid

hire_date
2023-02-10
2022-01-10
2021-04-19
2021-03-15
2020-07-22

#3. Employees not in Development department

Code:

```
select e.* from employees e

join departments d

on e.dept_id=d.dept_id
```

where d.dept_name !="Development";

Screenshot:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	email	phone	hire_date	dept_id	job_role
105	Karan Patel	karan@company.com	9090909090	2019-11-01	3	Accountant
102	Priya Singh	priya@company.com	9922334455	2021-03-15	1	HR
104	Neha Joshi	neha@company.com	9988776655	2023-02-10	4	Tester

[3]. Aggregate Functions

1. **Count employees in each department**

   Code:

   SELECT dept_id, COUNT(*) AS total_employees

   FROM employees

   GROUP BY dept_id;

   Screenshot:

Result Grid | Filter Rows:

dept_id	total_employees
1	1
2	3
3	1
4	1

2. **Avg salary of developers**

   Code:

   SELECT AVG(s.basic_salary) AS avg_salary

   FROM employees e

   JOIN salaries s ON e.emp_id = s.emp_id

WHERE e.job_role = 'Developer';

Screenshot:

Result Grid | Filter R

	avg_salary
▶	61000.000000

3. **Max salary in company**

Code:

SELECT MAX(basic_salary) AS max_salary

FROM salaries;

Screenshot:

Result Grid | Filter Rows:

	max_salary
▶	80000.00

[4]. GROUP BY + HAVING Clause

1. **Avg salary per department**

Code:

SELECT dept_id, AVG(basic_salary) AS avg_salary

FROM employees e

JOIN salaries s ON e.emp_id = s.emp_id

GROUP BY dept_id;

Screenshot:

Result Grid | Filter Rows:

dept_id	avg_salary
1	45000.000000
2	67333.333333
3	55000.000000
4	40000.000000

2. **Departments with more than 2 employees**

   Code:

   SELECT dept_id, COUNT(*) AS total_employees

   FROM employees

   GROUP BY dept_id

   HAVING COUNT(*) > 2;

   Screenshot:

Result Grid | Filter Rows:

dept_id	total_employees
2	3

[5]. Joins (All Types)

1. **Employees with departments**

   Code:

   SELECT e.emp_name, d.dept_name

   FROM employees e

   INNER JOIN departments d

ON e.dept_id = d.dept_id;

screenshot:

Result Grid | Filter Rows:

emp_name	dept_name
Amit Sharma	Development
Priya Singh	HR
Rahul Verma	Development
Neha Joshi	QA
Karan Patel	Finance

2. **Employees with their salary details**

Code:

SELECT e.emp_name, s.basic_salary, s.bonus, s.deductions

FROM employees e

INNER JOIN salaries s

ON e.emp_id = s.emp_id;

Screenshot:

Result Grid | Filter Rows: | Expo

emp_name	basic_salary	bonus	deductions
Amit Sharma	60000.00	5000.00	2000.00
Priya Singh	45000.00	3000.00	1000.00
Rahul Verma	80000.00	7000.00	5000.00
Neha Joshi	40000.00	2000.00	500.00
Karan Patel	55000.00	4000.00	1500.00

3. **Projects with assigned employees**

Code:

SELECT p.project_name, e.emp_name

FROM projects p

```
INNER JOIN employee_projects ep
ON p.project_id = ep.project_id
INNER JOIN employees e
ON ep.emp_id = e.emp_id;
```

Screenshot:

Result Grid | Filter Rows:

project_name	emp_name
Payroll System	Priya Singh
Payroll System	Amit Sharma
E-Commerce Portal	Neha Joshi
E-Commerce Portal	Rahul Verma
E-Commerce Portal	Amit Sharma

4. **Departments with no employees (RIGHT JOIN)**

Code:

```
SELECT d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

Screenshot:

dept_name
Management

5. **Full join alternative using UNION**

Code:

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id

UNION

SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Screenshot:

Result Grid | Filter Rows:

emp_name	dept_name
Amit Sharma	Development
Priya Singh	HR
Rahul Verma	Development
Neha Joshi	QA
Karan Patel	Finance

6. Subqueries/ Nested Queries

1. **Employees earning more than company avg**

CODE:

```
SELECT emp_name, basic_salary
FROM employees e
```

```
JOIN salaries s ON e.emp_id = s.emp_id

WHERE s.basic_salary > (SELECT AVG(basic_salary) FROM salaries);
```

Screenshot:

Result Grid | Filter Rows:

emp_name	basic_salary
Amit Sharma	60000.00
Rahul Verma	80000.00
Sonia Mehta	62000.00

2. **Highest earning employee**

Code:

```
SELECT emp_name, basic_salary

FROM employees e

JOIN salaries s ON e.emp_id = s.emp_id

WHERE s.basic_salary = (SELECT MAX(basic_salary) FROM salaries);
```

Screenshot:

Result Grid | Filter Rows:

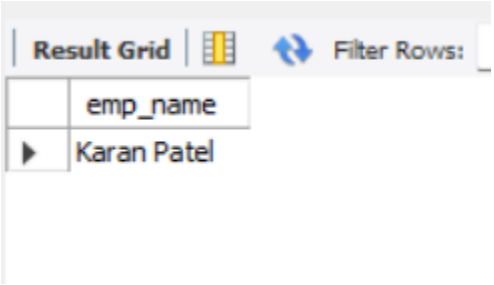
emp_name	basic_salary
Rahul Verma	80000.00

3. **Employees not assigned to any project**

Code:

```
SELECT emp_name
FROM employees e
WHERE emp_id NOT IN (SELECT emp_id FROM employee_projects);
```

Screenshot:



emp_name
Karan Patel

7. Nested Subqueries

1. Second highest salary

Code:

```
SELECT emp_name, basic_salary
FROM employees e
JOIN salaries s ON e.emp_id = s.emp_id
WHERE s.basic_salary = (
    SELECT MAX(basic_salary)
    FROM salaries
    WHERE basic_salary < (SELECT MAX(basic_salary) FROM salaries)
);
```

Screenshot:

Result Grid | Filter Rows:

	emp_name	basic_salary
▶	Sonia Mehta	62000.00

2. **Projects with more than 2 employees**

   Code:

```
SELECT project_name
FROM projects p
WHERE project_id IN (
    SELECT project_id
    FROM employee_projects
    GROUP BY project_id
    HAVING COUNT(emp_id) > 2
);
```

   Screenshot:

Result Grid | Filter Rows:

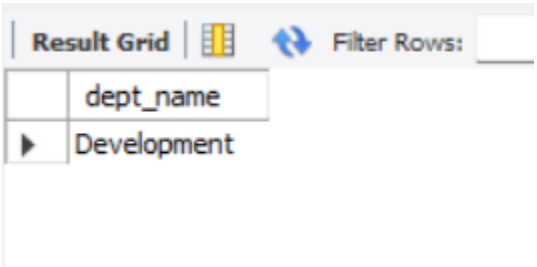
	project_name
▶	E-Commerce Portal

3. **Departments having avg salary > finance**

Code:

```
SELECT d.dept_name

FROM departments d

JOIN employees e ON d.dept_id = e.dept_id

JOIN salaries s ON e.emp_id = s.emp_id

GROUP BY d.dept_id, d.dept_name

HAVING AVG(s.basic_salary) > (

   SELECT AVG(s.basic_salary)

   FROM employees e

   JOIN salaries s ON e.emp_id = s.emp_id

   WHERE e.dept_id = (SELECT dept_id FROM departments WHERE
dept_name = 'Finance')

);
```

Screenshot:



[9]. Views

1. High earners

```
CREATE VIEW high_earners AS

SELECT emp_id, basic_salary

FROM salaries
```

WHERE basic_salary > 60000;

Screenshot:

14	18:17:44	CREATE VIEW high_earners AS SELECT emp_...	0 row(s) affected	0.015 sec

2. Department-wise salary summary

code:

```
CREATE VIEW dept_salary_summary AS

SELECT d.dept_name, AVG(s.basic_salary) AS avg_salary

FROM employees e

JOIN salaries s ON e.emp_id = s.emp_id

JOIN departments d ON d.dept_id = e.dept_id

GROUP BY d.dept_name;
```

Screenshot:

15	18:19:35	CREATE VIEW dept_salary_summary AS SELE...	0 row(s) affected	0.016 sec

3. Employee project allocation

code:

```
CREATE VIEW employee_project_view AS

SELECT e.emp_name, p.project_name, ep.role

FROM employees e

JOIN employee_projects ep ON e.emp_id = ep.emp_id

JOIN projects p ON p.project_id = ep.project_id;
```

Screenshot:

16	18:22:36	CREATE VIEW employee_project_view AS SEL...	0 row(s) affected	0.015 sec

Summary of Learnings

1. Learned to design a normalized database for HR & Payroll with multiple tables and relationships.

2. Applied DDL commands to create tables with constraints like PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK.

3. Practiced DML operations to insert, update, and delete employee, department, project, and salary data.

4. Explored various JOINs (INNER, LEFT, RIGHT, FULL) to combine data across multiple tables.

5. Used aggregate functions (SUM, AVG, COUNT, MAX, MIN) along with GROUP BY and HAVING clauses for analysis.

6. Created subqueries and nested queries to solve real-world problems like highest salary, second highest salary, and employees without projects.

7. Handled many-to-many relationships using a linking table for employees and projects.

8. Practiced views and stored procedures to simplify repetitive queries and automate tasks.

9. Developed skills to perform analytical reporting, like identifying departments with high average salary or projects with multiple employees.

10. Gained hands-on experience in building end-to-end SQL solutions for HR decision-making and payroll management.